Smart EV Delivery System Using A* with Battery Constraint

Project submitted to the SASTRA Deemed to be University
in partial fulfilment of the requirements
for the award of the degree of

B. Tech. Robotics and Artificial Intelligence

Submitted by

Munaga Asritha
(Reg. No.: 128179033)

SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING
THANJAVUR, TAMIL NADU - 613401

SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)
THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING
THANJAVUR, TAMIL NADU - 613401

Bonafide Certificate

This is to certify that the project based assessment report titled **"Smart EV Delivery System Using A* with Battery Constraint"** submitted in partial fulfilment of the requirements for the award of the degree of B. Tech, Robotics & Artificial Intelligence Engineering to the SASTRA Deemed to be University, is a bona-fide record of the work done by **Ms. Munaga Asritha** (Reg. No. 128179033) during the IV semester of the academic year 2025-2026, in the **School of Electrical & Electronics Engineering**, under my supervision for the course titled Introduction to Artificial Intelligence (EIE115R01).

Course Co-ordinator

Name with Affiliation : Dr. K. Ghousiya Begum, Assistant Professor III

Date : _____

Project Viva-voce held on: _____

TABLE OF CONTENTS

SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)
THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING
THANJAVUR – 613 401

Declaration

I declare that the report titled **"Smart EV Delivery System Using A* with Battery Constraint"** submitted by me is an original work done by me under the supervision of Dr. K. Ghousiya Begum, Associate Professor III, School of Electrical & Electronics Engineering, SASTRA Deemed to be University during the fourth semester of the academic year 2025-26. The work is original and wherever I have used materials from other sources, I have given duew credit and cited them in the text of the report.

Signature of the candidate : _____

Name of the candidate : Munaga Asritha (Reg.No. 128179033, B.Tech RAI)

Date : _____

Acknowledgements

I would like to thank our Honourable Chancellor **Prof. R.Sethuraan** for providing me with an opportunity and the necessary infrastructure for carrying out this project as a part of my curriculum.

I would like to thank our Vice- Chancellor **Dr. S.Vaidhyasubramaniam** and **Dr. S.Swaminathan**, Dean, Planning & Development, for the encouragement and strategic support at every step of our college life.

I extend my sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

I extend my heartfelt thanks to **Dr. K. Thenmozhi**, Dean, School of Electrical and Electronics Engineering ,**Dr. Sridhar K**, Dean, Research, **Dr. V. Muthubalan**, Associate Dean, Reasearch, **Dr. A. Krishnamoorthy**, Associate Dean, Academics, **Dr. Manigandan N.S.**, Associate Dean, Infrastructure, **Lt. Dr. Vijayarekha K**, Associate Dean, Students Welfare.

Dr. K. Ghousiya begum, my course coordinator and Associate Professor – III at the School of Electrical and Electronics Engineering, was the driving force behind my project idea. Her expertise and advice were incredibly helpful throughout.

Abstract

This project presents a Smart Electric Vehicle(EV) Delivery System simulated using Python and Pygame library. The system simulates a grid-based environment with warehouses, chargers, roads, and houses where packages need to be delivered. The A* algorithm is used to find the shortest and best path to their destination.

Each EV continuously checks its battery and automatically reroutes to the charging station when battery is low. The system uses a prority-based order management system where orders are divided into Express, Standard, and Scheduled. First, Express orders are delivered with highest priority, then Standard and Scheduled orders are delivered only at the mentioned time comes.

The simulation works in two modes: auto and manual mode. In the manual mode, users are allowed to select delivery locations and order types. It also includes features like batching and alternative node selection when no direct path is avialable. A visual dashboard displays EV battery levels, current status of EVs, current mode and pending orders.

Keywords: Electric Vehicle, A* Pathfinding, Priority Scheduling, Battery Management, Multi-Agent Systems, Pygame

CHAPTER 1 INTRODUCTION

The use of electric vehicles (EVs) in transport sector has been increasing in past few years. In urban areas, there is a need of speed and efficient delivery service, but also environment friendly. EVs are a better alternative to fuel-based vehicles because they reduce pollution and are suitable for short-distance deliveries within cities.

Traditional delivery systems use fixed routes, which many not suitable for dynamic environments where delivery conditions and order priorities change frequently. An intelligent delivery system must be capable of making real-time decisions such as selecting the best route, managing energy consumption, and handling multiple orders simultaneously.

This project presents a *Smart EV Delivery System*, a simulation designed to model how electric vehicles can be used for package delivery in a city environment. The system is built using Python and the Pygame library. This project uses a grid-based map which consists of warehouses, charging stations, roads and houses. EVs move through this environment to pick up and deliver orders while managing their battery levels.

To improve delivery efficiency, this project uses techniques such as A* pathfinding for optimal routing, priority-based order management, and coordination among multiple EVs. These methods help in making better routing. The A* algorithm finds the shortest available path for each EV on the grid, while the priority system ensures that urgent orders are handled before less priority ones.

In this system, the city map is generated automatically and three EVs operate within it. Each EV can take delivery orders, find the shortest path, monitor battery level and go for charging when battery falls below a threshold. Orders are divided into three types: Express, Standard, and Scheduled, based on their urgency and delivery time.

This project also involves other features such as batch ordering, where an EV can carry up to five orders in one trip, task assignment, and visualization. A visual dashboard built using Pygame displays the battery levels, current states, and order queues of all EVs, making it easy to monitor the system.

Through this simulation, the project shows that by implementing intelligent routing and energy-aware decisions, the overall efficiency of the EV-based delivery systems can be improved.

1.1 Objectives

- To design and implement a multi-agent EV delivery simulation with battery constraints.
- To apply A* algorithm for finding the shortest and best delivery routes.
- To implement a priority based order management system for Express, Standard and Scheduled deliveries.

- To simulate battery usage and automatic charging.
- To build an interactive visualization using Pygame library.
- To evaluate system performance through real-time simulation and status monitoring.
- To optimize delivery efficiency using techniques such as order batching and task assignment.

1.2 Scope of the Project

The scope of this project is limited to a simulated two-dimensional city environment. The system does not interact with real EVs or live map data. The city grid is generated automatically for each simulation, so the layout may vary in different runs. EV agents navigate using the available road network, and battery usage is calculated based on the distance travelled by each vehicle.

The project focuses on demonstrating key concepts such as path optimization, priority-based scheduling, and battery-aware routing in a controlled environment. However, Features such as real traffic conditions, vehicle load-based battery consumption, and communication between EVs are not considered in this project.

CHAPTER 2 METHODOLOGY

2.1 System Architecture Overview

The simulation is built entirely in Python, using the Pygame library. The code is split into six modules config.py, world.py, orders.py, ev.py, and ui.py. main.py runs the simulation updating the time, taking inputs from the user and updating all EV agents,

Each agent works individually. It moves between the following states: idle, loading, delivering, returning and charging based on conditions like new orders or battery level. Figure 2.1 illustrates the state transition diagram of an EV agent.

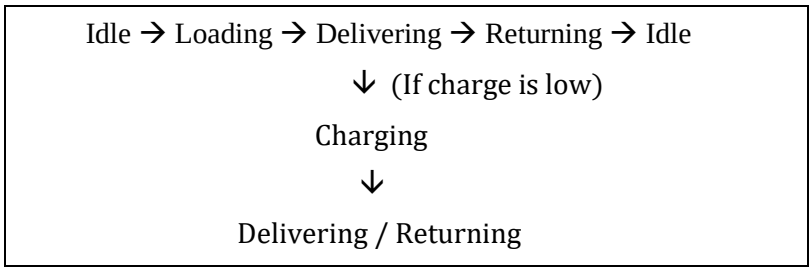

Fig 2.1 — EV Agent State Transition Diagram

2.2 Map Generation and Grid Layout

The city is a grid of small squares called cells. Each cell is 28 pixels wide. The window size is 1280 x 790 pixels.

When the program starts, horizontal roads are drawn at every other row and vertical roads drawn after some interval. After that, four warehouses and five charging stations are added to the cells adjacent to roads. Lastly, houses are placed on all the cells near roads. Each house gets a number label. The cell types are: 0 = empty land, 1 = road, 2 = warehouse, 3 = charging station, 4 = house. The EVs can only travel on cells of type 1, 2, or 3. Table 2.1 summarises the key simulation configuration parameters used in this system.

Parameter	**Value**
Grid : Width × Height	1280 × 790 px
Cell Size	28 px
Max Battery	100 units
Low Battery Threshold	25 units
EV Speed	8 frames/step
Batch Size	5 orders
Number of EVs	3
Simulation Frame Rate	30 fps

Table 2.1 — Simulation Configuration Parameters

Each cell carries a type code that determines how EVs interact with it, as described in Table 2.2.

Code	Cell Type	EV Can Travel On
0	Empty land	No
1	Road	Yes
2	Warehouse	Yes
3	Charging station	Yes
4	House (delivery point)	No (stop adjacent)

Table 2.2 — Cell Type Codes

2.3 A* Pathfinding Algorithm

The A* algorithm is used by EVs to find the shortest path. Whenever an EV needs to move to a house, charger, or warehouse it calls astar() function.

The function takes a start and goal cell and returns the path. It is implemented using heapq. The value of each cell is calculated using:

Total estimated cost f is calculated as

$$f(n) = g(n) + h(n)$$

where, g (n) is the total distance travelled from start to n, and h (n) is the Manhattan distance from n to goal. Manhattan distance is used because movement is only in four directions (only up, down, left, right directions and diagonal movement not allowed). If no path is found, a blocked-route dialog is displayed, nearby road nodes are suggested as alternate delivery points.

2.4 EV Battery Management

Each EV has a battery that decreases as it moves (1 unit per cell). At every step, EV checks its battery level. If battery is lower than threshold (25 units), it goes to charge. Figure 2.2 shows the battery decision flowchart followed by each EV.

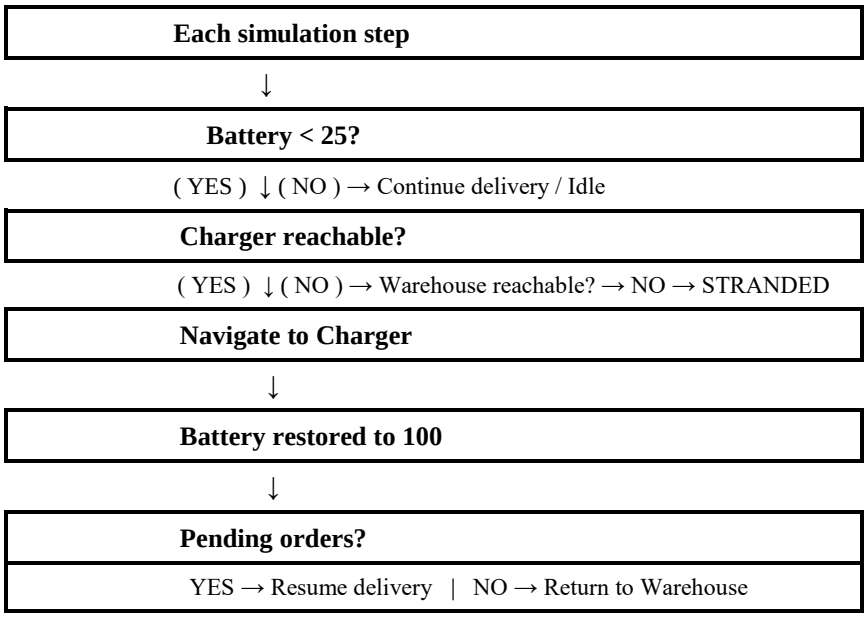

Fig 2.2 — Battery Decision Flowchart

If neither is reachable, the EV becomes stranded: Its orders are returned back to the queue and assigned to other EVs. When the EV reaches a charger, the battery is restored. After charging, it continues delivery if orders are still pending.

2.5 Delivery Mode Classification

Orders are categorized into three delivery modes, as listed in Table 2.2. They are :

Mode	Colour	Priority(Base Value)
Express	Red	1 (Highest)
Standard	Green	2 (Normal)
Scheduled	Blue	0 (when due), 10 (before due)

Table 2.2 — Delivery Mode Attributes

Express orders are delivered first. Standard orders follow normal queue order and gain priority over time. Scheduled orders are delivered only after their specified time, after which they are treated like high-priority orders. In auto mode, delivery types are selected randomly.

2.6 Order Management System

Orders are stored in a list called order_queue(). Each order contains the House location, Delivery type, order ID, the frame it was created, and for Scheduled orders, the target delivery frame. Orders are created using make_order() and added to the queue. The queue is sorted based on priority. Scheduled orders are not considered for delivery until their time is reached. Once the time arrives, they are treated like high-priority orders.

Orders use aging factor. If an order waits too long, its priority improves over time. If a standard order waits too long gains priority faster than other orders, so they are not get delayed.

When an EV becomes free, it selects the top order from the queue. It groups nearby orders up to 5 and loads in one batch. Then the selected orders are removed from the queue.

2.7 Simulation and Visualisation

The simulation runs at 30fps using Pygame. In each frame, the system processes user input, update all EVs, and redraws the screen. Screen is divided into two parts: Top part - city map, Bottom part – status panel.

EVs are shown as coloured circles with their number. Their paths are shown as lines of the same colour, and delivery locations are marked using coloured pins.

Bottom panel shows details of each EV, including battery level, loaded orders, current state. It shows the current mode, queue size, upcoming orders.

A startup screen is displayed, that allows the user to choose mode. A simulation clock is displayed in the top right corner.

CHAPTER 3 RESULTS AND DISCUSSION

3.1 Startup Screen

When simulation starts, a startup screen is displayed. It allows user to choose the mode. The screen shows a legend explaining EV colours, Delivery types, Map elements. Figure 3.1 shows the startup screen as seen when the simulation is launched.

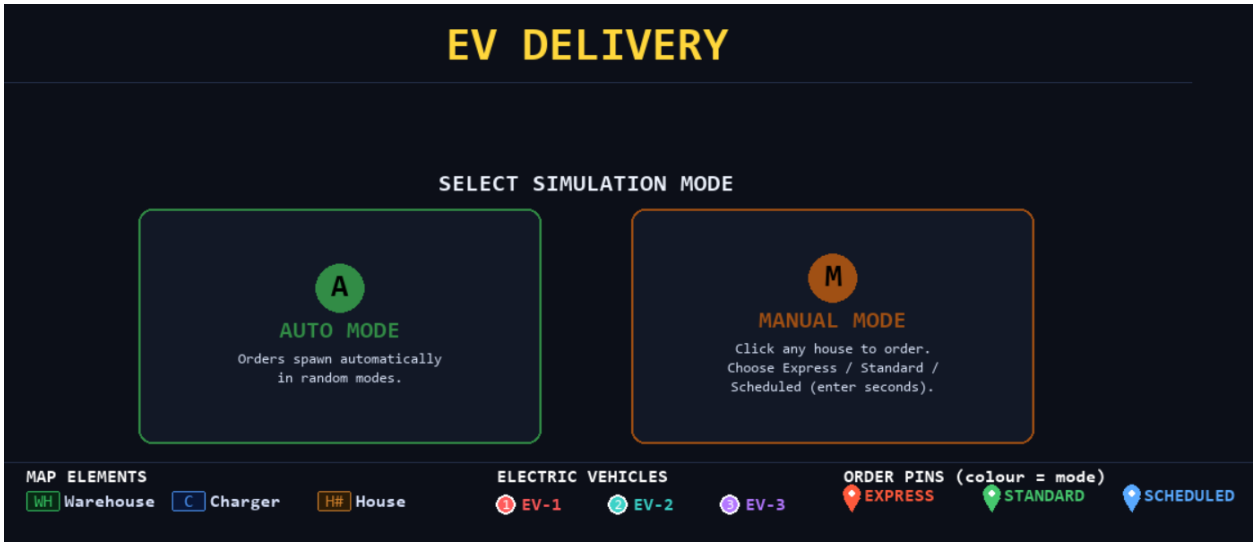

Fig 3.1 Startup Screen

3.2 Map Visualization

The project environment is represented as grid-based city consisting of roads, houses, chargers, warehouses. Warehouses act as starting or return points for EVs, Houses act as delivery locations. Charging stations are placed near roads to support battery recharging. Figure 3.2 presents the full map view of the simulated city environment.

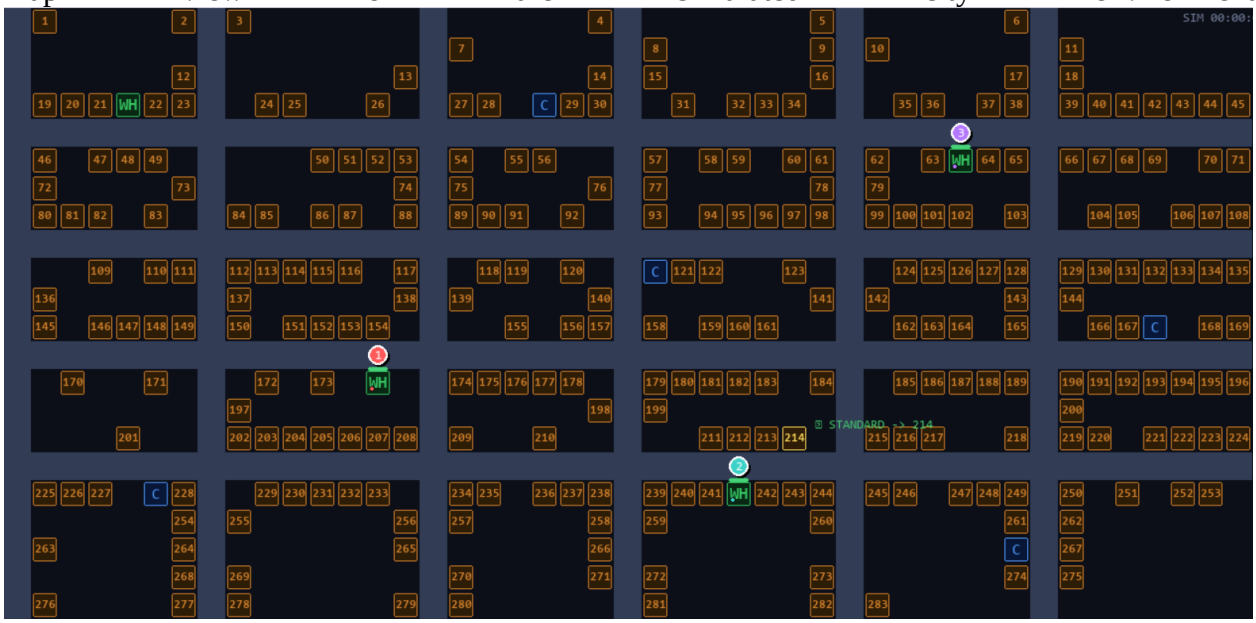

Fig 3.2 Full Map View

3.3 UI Display and Bottom Panel Interaction

The simulation interface is divided into two sections: the top section displays the city map, and the bottom panel. The bottom panel displays important details like the EV's status, battery status, current operation mode, and order queue. It enables the operator to observe what is going on in the system and see how the orders are being handled.

New orders are assigned to EVs depending on the order queue. Depending on the priority of the order and the EV's battery status, an EV modifies its route for efficiency. Figure 3.3 displays the UI with the bottom panel showing EV status, battery levels, and the order queue.

Fig 3.3 UI displaying and bottom panel

3.4 Active Delivery Simulation

Once the simulation begins, EVs start handling deliveries. The path of EVs and delivery updates are clearly shown. Orders appear on houses as coloured pins where that colour represent the type of order and gets added to queue. EV picks orders from the queue and deliver them based on prirority. Figure 3.4 shows three EVs simultaneously handling deliveries across the city grid.

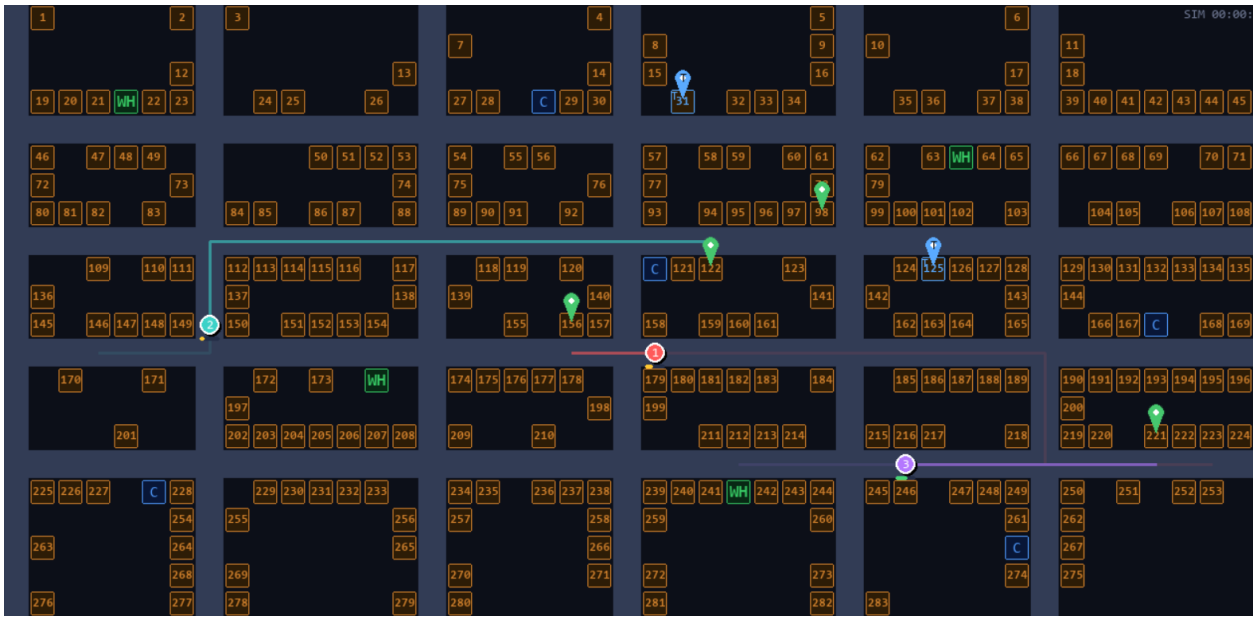

Fig 3.4 Three EVs Handling Delivery

3.5 Battery and Charging Behaviour

Each EV continuously monitors its battery level during its operation. When battery level goes below threshold value, EV automatically moves to charging station to recharge. After charging, if there are pending deliveries then resumes delivery. Figure 3.5 illustrates an EV navigating to a charging station when its battery falls below the threshold.

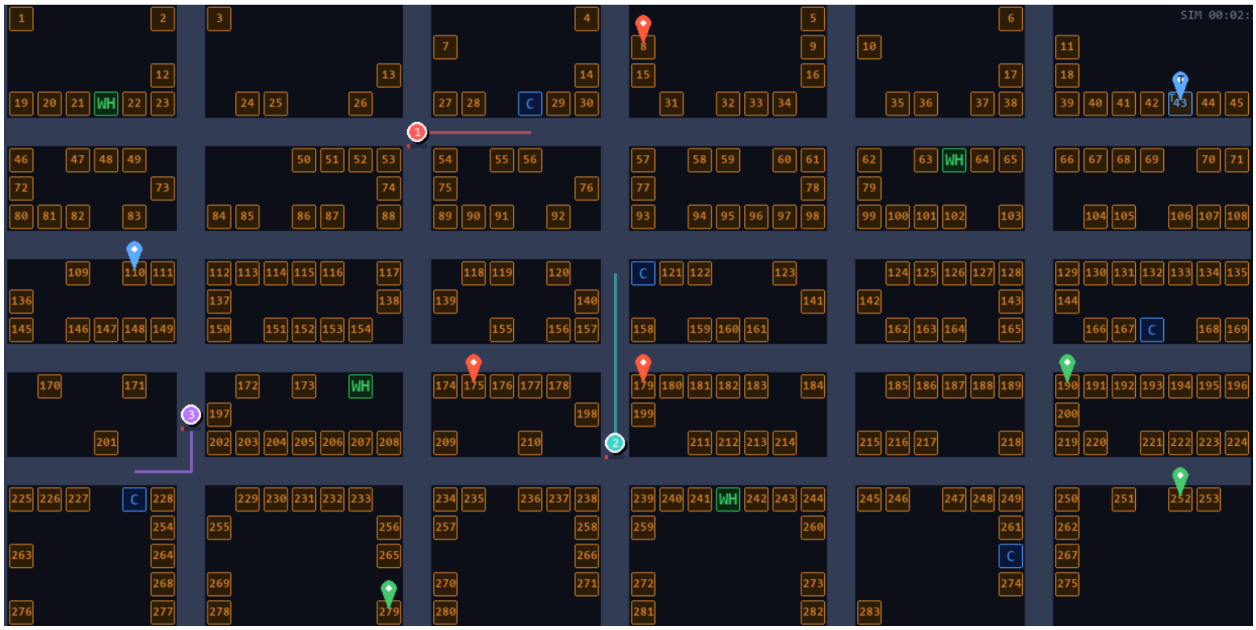

Fig 3.5: EV moving to charging station when battery level falls below threshold

3.6 Manual Mode and User Interaction

In manual mode, User can select houses to create orders. The user can choose the delivery type. For scheduled deliveries, a dialog box is displayed where the user enters the desired delivery delay in seconds. Figure 3.6 shows the dialog box used for entering the scheduled delivery time.

EVs adjust their routes automatically based on new orders, considering priority and battery constraints.

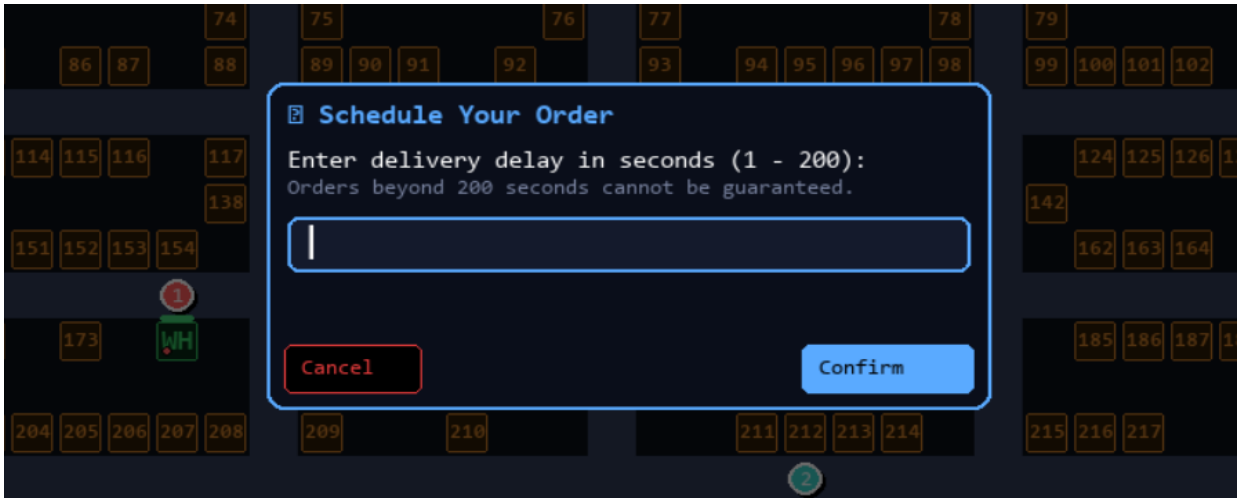

Fig 3.6 Dialog box for entering scheduled delivery time

3.7 Blocked Route Handling

If an EV cannot find a path to a destination, a blocked route dialog box is displayed to allows the user to select any alternative node nearby for delivery. This ensures delivery continues even when direct path is not available. Figure 3.7 shows the blocked route dialog with alternative node selection options.

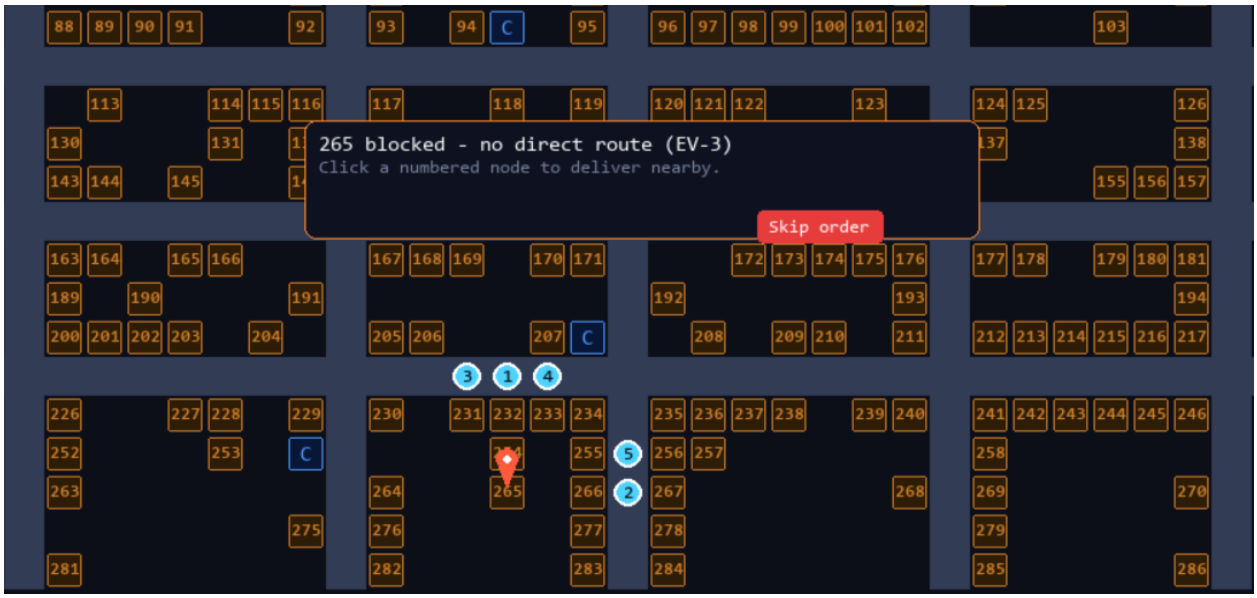

Fig 3.7 Blocked route handling with alternative node selection dialog

3.8 Performance Summary

The following table summarises observed metrics across a typical 5-minute simulation run in Auto Mode:

Metric	EV 1	EV 2	EV 3
Orders Completed	28	26	24
Charging Events	15	12	12
Stranding Events	0	0	0
Blocked Routes Encountered	0	1	0
Average Delivery Time (frames)	~142	~138	~145
Battery Depleted Events	0	0	0

CHAPTER 4 CONCLUSIONS AND FUTURE SCOPE

4.1 Conclusions

This project successfully demonstrates a Smart EV Delivery System through a grid-based simulation built using Python and Pygame. In this system, The simulation simulates a realistic city setting wherein three electric vehicles function together in carrying out deliveries of packages from the warehouse to the correct houses according to order priority and battery capacity.

The A* pathfinding algorithm efficiently computes optimal routes for EV navigation using Manhattan distance as the heuristic. The system is capable of handling blocked routes by allowing alternative node selection, ensuring uninterrupted delivery operations.

The priority-based order management system ensures that Express orders are delivered first, followed by Standard and Scheduled orders. The aging mechanism prevents long waiting times by gradually increasing the priority of older orders.

Battery management is handled effectively by continuously monitoring the battery level of each EV. When the battery falls below a threshold, EVs automatically go to charging stations and resume delivery after charging. In cases where charging is not possible, orders are reassigned to other EVs. Additionally, The use of batching reduces travel time and distance by allowing multiple deliveries at a time.

Through this project, I understood real-world problems like delivery routing, battery management and how EV based delivery system can be designed and managed in a city environment.

4.2 Future Scope

This project can be further improved in several ways:

- Integration with real-world maps using OpenStreetMap (OSM)
- Improving the battery model by considering factors like load
- Adding communication between EVs
- Using advanced algorithms and integration with RL for better delivery efficiency
- Addition dynamic traffic conditions, obstacles and road blockages to look realistic

CHAPTER 5 REFERENCES

1. V. P. Damle and S. Susan, "Dynamic Algorithm for Path Planning Using A-Star With Distance Constraint," in *Proc. 2nd International Conference on Intelligent Technologies (CONIT)*, Hubli, India, pp. 1–6, Jun. 2022.
2. J.-H. Ko and H.-W. Jang, "Battery-Aware Routing for Electric Vehicles Using Real-Time Charging Station Data," *IEEE Access*, vol. 9, pp. 112034–112048, 2021.
3. P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, Jul. 1968.